# Bit-Pop: Ultra-Fast Multi-Genome DNA Read Classification with Fuzzy K-mer Matching and EM Refinement

Mladen Popović
*Independent Researcher*
mladenpop@gmail.com
https://github.com/mladenpop-oss/bit-pop

May 10, 2026

**Abstract**

We present Bit-Pop, a high-performance tool for multi-genome DNA read classification that identifies which genome in a collection best matches each sequencing read. While existing aligners such as Bowtie2, BWA-MEM, and minimap2 map reads to single reference genomes, Bit-Pop addresses the complementary problem of simultaneous classification across collections of $N$ genomes. The pipeline combines four key stages: (1) an FM-index built via the SA-IS algorithm (libsais) for efficient k-mer lookup; (2) a top-$N$ rarest k-mer anchor filter with quality-aware filtering and reverse complement support; (3) a 2-bit XOR alignment achieving approximately 2.3 ns per 31-base chunk, with Myers edit distance as a fast alternative to Smith–Waterman (23–54× faster); and (4) a combined ranking formula balancing alignment score (85%) and k-mer rarity (15%). For difficult cases, Bit-Pop provides three fuzzy k-mer matching methods (fuzzy-kmer, fuzzy-seed, neighborhood) and an Expectation-Maximization (EM) post-processing algorithm for multi-candidate refinement. Benchmarked on simulated Illumina-like reads (0.1%–10% error rate, 100 bp) across three distantly related genomes (*Escherichia coli*, *Staphylococcus aureus*, *Saccharomyces cerevisiae*), Bit-Pop achieves 99.3% mapping rate and 99.9% classification accuracy in 0.9s for 20,000 reads. On the CAMI Low Complexity benchmark with 62 microbial genomes (20,000 paired-end reads, 2×150 bp), Bit-Pop achieves 92.0% mapping rate and 85.9% overall accuracy. The evo_* (>99.9% identical strains) remain the most challenging subset at 59.8% accuracy (with EM refinement), representing a fundamental information-theoretic limitation of short-read k-mer-based classification. Bit-Pop is designed as a focused tool for species classification, contamination detection, and targeted metagenomic analysis where users work with known genome collections.

**Availability:** Source code at `https://github.com/mladenpop-oss/bit-pop` (MIT License). DOI: `https://doi.org/10.5281/zenodo.20043593`.

## 1 Introduction

The cost of DNA sequencing has continued to decline over the past decade, enabling applications ranging from clinical metagenomics to environmental microbiome profiling. In many of these applications, a fundamental question arises: *which organism does this read come from?* When researchers work with a curated collection of reference genomes—whether tracking pathogen strains in a hospital, monitoring contamination in a bioproduction facility, or profiling a defined microbiome—efficiently assigning each read to its source genome becomes a critical computational task.

Existing short-read aligners—Bowtie2 [Langmead and Salzberg, 2012], BWA-MEM [Li, 2013], and minimap2 [Li, 2018]—are optimized for mapping reads against a single reference genome or a pangenome graph. These tools achieve remarkable speed and accuracy but do not natively address the multi-genome classification problem. While one could theoretically index all target genomes into a single reference and map reads against it, this approach conflates two distinct tasks: (1) finding the optimal genomic position for each read, and (2) determining which genome in a collection best explains each read.

Tools specifically designed for taxonomic classification, such as Kraken2 [Wood et al., 2019] and Centrifuge [Kim et al., 2016], take a different approach: they build massive k-mer databases covering entire taxonomic

databases (e.g., NCBI nt/nr), enabling broad-spectrum classification. However, these tools require large databases (100 GB+), internet connectivity for updates, and hours to days to build. They are designed for "unknown unknown" scenarios where you do not know what you are looking for.

Bit-Pop was designed for the complementary use case: **targeted classification** where users know their genomes of interest. Given a curated collection of $N$ genomes (potentially hundreds or thousands) and a set of sequencing reads, Bit-Pop maps each read against all genomes simultaneously and returns a ranked list identifying the most likely source genome, along with alignment position, score, and contextual flanking sequence.

The key design principles are:

- **Compact indexing:** All reference genomes are indexed in a single FM-index structure, with database sizes proportional only to the user's genomes (MBs, not GBs).
- **Speed via bit-level parallelism:** DNA bases are encoded in 2 bits, enabling alignment of up to 31 bases per CPU word through XOR comparison—achieving approximately 2.3 ns per 31-base XOR chunk.
- **Multi-stage pipeline:** Anchor-based k-mer filtering, fast XOR alignment, Myers edit distance, and Smith–Waterman refinement are combined for optimal speed-accuracy trade-off.
- **Fuzzy matching for strain resolution:** Three fuzzy k-mer methods (fuzzy-kmer, fuzzy-seed, neighborhood) address the challenge of highly similar strains (>99.9% identity).
- **EM post-processing:** An Expectation-Maximization algorithm refines multi-candidate mappings using population-level abundance signals.
- **Focused scope:** Bit-Pop is not intended to replace general-purpose aligners but to solve a specific problem—species classification across known genome collections—that existing tools do not address directly.

This paper describes the algorithms, implementation, and benchmark results of Bit-Pop, demonstrating its feasibility for species-level read classification across collections of multiple bacterial and eukaryotic genomes, including the challenging CAMI Low Complexity benchmark with 62 microbial genomes.

# 2 Methods

## 2.1 FM-Index Construction

The FM-index [Burrows and Wheeler, 1994, Ferrada and Grossini, 2007] is a compressed full-text substring index built on the Burrows–Wheeler Transform (BWT) of the concatenated reference collection. Bit-Pop constructs the FM-index using the SA-IS algorithm [Kärkkäinen and Sanders, 2003], implemented via the libsais library [Sósten, 2020–2024], which achieves linear-time $O(L)$ construction where $L$ is the total length of all genomes.

Given $N$ genomes with total length $L$, the BWT is constructed by:

1. Encoding bases as $u8$ values: \$=0, A=1, C=2, G=3, T=4.
2. Concatenating all genome sequences with unique separator characters.
3. Computing the suffix array via SA-IS (linear time).
4. Deriving the BWT from the suffix array.
5. Building the Occ (occurrence) counter array for backward search with rank sampling (interval=32).

The SA construction using SA-IS operates in $O(L)$ time and space, significantly faster than the previous radix-sort prefix doubling approach. The BWT and Occ array together occupy approximately $2L$ bytes (2 bits per base for BWT, plus 8 bits per position for Occ counters with periodic sampling).

Persisted indexes are stored using `memmap2` for memory-mapped file access and zstd compression for the BWT, achieving typical compression ratios of 3–4:1. The SA is stored uncompressed to enable $O(1)$ random access during mapping.

## 2.2 Anchor-Based K-mer Filtering

The first stage of mapping uses a top-$N$ anchor-based filter to identify candidate positions across all indexed genomes. Given a read of length $R$, the algorithm:

1. Extracts all overlapping k-mers (configurable, default $k = 10$ for benchmarks) from the read.
2. For each k-mer, queries the FM-index to count its total occurrences across all genomes using backward search.
3. Selects the **top-$N$ rarest** k-mers as anchors—those with fewest total positions—providing fallback candidates when the primary anchor contains sequencing errors.
4. For each anchor k-mer, retrieves all positions via backward search.
5. Applies smart stride sampling: if more than 100 anchor positions are found, they are subsampled with stride $\lceil n/100 \rceil$ to avoid redundant close alignments.
6. Skips highly repetitive k-mers exceeding a configurable threshold (default $10^4$ total occurrences).

This approach is significantly more efficient than iterating all k-mers in the read when the genome collection contains many repetitive elements. The top-$N$ rarest k-mer anchors minimize false negatives caused by sequencing errors in the anchor k-mer while maintaining high discriminative power. The recommended setting is $N = 2$ for balance between speed and accuracy; $N = 3$ provides +1.4% mapping rate at $\sim 3\times$ computation time.

Quality-aware filtering further refines this stage: k-mers containing bases with Phred quality below a minimum threshold (default Q20) [Ewing and Green, 1998] are excluded from anchor selection, reducing false positives in low-quality regions.

## 2.3 2-Bit XOR Alignment

For each candidate position identified by the anchor filter, Bit-Pop performs 2-bit XOR alignment to score the read against the genomic region. DNA bases are encoded as 2-bit values (A=00, C=01, G=10, T=11), allowing a read of up to 31 bases to be packed into a single 64-bit word.

Given a packed read $P$ and a genomic window $W$ of the same length:

$$\text{XOR} = P \oplus W$$

Each 2-bit field in the XOR result is zero if bases match and non-zero if they mismatch. The alignment score is:

$$\text{score} = \frac{\text{matches}}{\text{read\_length}} = \frac{R - \sum_{i=0}^{R-1} [\text{xor}_i \neq 0]}{R}$$

This operation requires a single XOR instruction and a population-count equivalent, achieving approximately 2.3 ns per 31-base XOR chunk on modern CPUs—orders of magnitude faster than traditional dynamic programming alignment.

For reads exceeding 31 bases, the XOR alignment is applied in chunks of 31 bases, with the final score computed as the average chunk score. While this approach does not handle gaps (indels), it provides excellent discrimination for high-quality reads where mismatches are the primary source of disagreement.

## 2.4 Myers Edit Distance

For reads with moderate XOR confidence scores, Bit-Pop applies Myers' bit-vector algorithm for edit distance computation. Myers' algorithm computes the exact edit distance between a pattern and text using bit-parallel operations, achieving 23–54× speedup over Smith–Waterman while providing exact (not approximate) edit distance [Myers, 1999].

This provides a fast intermediate between the approximate XOR alignment and the full Smith–Waterman refinement, enabling Bit-Pop to handle reads with indels without the full $O(m \times n)$ cost of SW.

## 2.5 Smith–Waterman Refinement

For reads with low XOR confidence scores ($< 0.9$), Bit-Pop applies Smith–Waterman (SW) local alignment as a refinement step. The Smith–Waterman algorithm [Smith and Waterman, 1981] uses standard scoring parameters (match=+2, mismatch=-1, gap open=-2) and full traceback to generate CIGAR strings with indel operations (M, I, D).

For reads exceeding 31 bases, Bit-Pop employs **chunked Smith-Waterman**: the read is split into 31-base chunks, each aligned independently against the candidate genomic region using full SW with traceback. The per-chunk CIGAR operation codes are concatenated and collapsed into a final CIGAR string.

## 2.6 Quality-Aware Scoring

Bit-Pop supports Phred-scaled quality-aware scoring at multiple stages:

**Quality-filtered k-mer selection:** During anchor selection, k-mers containing bases below a minimum Phred quality threshold (default Q20) are excluded from anchor selection.

**Quality-aware XOR alignment:** Mismatches at high-quality positions receive larger penalties:

$$\text{adjusted\_score} = \frac{\text{matches}}{R} + \sum_{i \in \text{mismatches}} \left( -\frac{Q_i}{20} \right)$$

**Quality-aware Smith-Waterman:** The SW scoring matrix incorporates per-base quality penalties:

$$\text{mismatch\_penalty}_i = -\min\left( \frac{Q_i}{10}, 5 \right)$$

## 2.7 Reverse Complement Support

Bit-Pop evaluates both forward and reverse complement orientations for each read. The reverse complement is computed at the 2-bit encoding level (A↔T, C↔G, then reverse), and the best alignment (forward or RC) is selected. SAM FLAG 0x10 (REVERSE) is set when the RC alignment wins, achieving approximately 1.1% RC usage in benchmarks.

## 2.8 Multi-Genome Ranking

After scoring each read against all indexed genomes, Bit-Pop applies a combined ranking formula:

$$\text{combined\_score} = \alpha \cdot \text{align\_score} + (1 - \alpha) \cdot \text{rarity}$$

with $\alpha = 0.85$. The alignment score reflects the quality of the best match against a genome, while rarity provides a modest boost to matches in genome-specific regions:

$$\text{rarity} = \frac{1}{\max(1, \text{occurrences\_of\_first\_k\_mer})}$$

## 2.9 Fuzzy K-mer Matching

For highly similar genomes (>99.9% identity), exact k-mer matching is insufficient because most k-mers are shared between strains. Bit-Pop provides three fuzzy k-mer matching methods:

**Fuzzy k-mer:** Generates all possible k-mer variants with $N$ substitutions and queries the FM-index for each variant. This provides the best accuracy for strain resolution but is $\sim$30$\times$ slower than exact matching ($N = 1$).

**Fuzzy seed:** Allows $N$ mismatches in spaced seed "match" positions. Spaced seeds use a pattern of "match" and "don't care" positions (e.g., 111010111011), reducing the number of variants to generate while maintaining sensitivity. This provides a good balance between accuracy and speed, $\sim$20$\times$ slower ($N = 1$).

**Neighborhood:** Builds a hash table at index build time for all k-mer neighborhoods (k-mers within $N$ mismatches). This provides $O(1)$ fuzzy lookup at query time but increases index size by $\sim$60$\times$ ($N = 1$).

### 2.10 EM Post-Processing

The Expectation-Maximization (EM) algorithm refines multi-candidate SAM mappings by using population-level abundance signals. When a read maps to multiple genomes with similar scores, EM iteratively:

1. **E-step:** Estimates genome abundances based on current read assignments (soft assignments using softmax with temperature parameter).
2. **M-step:** Reassigns reads to maximize the likelihood given estimated abundances.

The algorithm typically converges in 9–11 iterations. Key parameters:

- **Temperature:** Controls softness of assignments (default: 0.1). Lower temperature = harder assignments.
- **Top-$K$:** Number of top candidates per read (default: 30).
- **Convergence:** KL divergence threshold for stopping (default: 0.001).
- **Confidence threshold:** Only applies EM to reads with ambiguous assignments (default: 0.95).

On the CAMI benchmark, EM provides +1.4pp improvement on evo_* (>99.9% identical strains) and +0.6pp overall improvement. However, detailed analysis shows EM fixes 805 wrong predictions but breaks 755 correct ones (net +50), confirming that the fundamental limitation is information-theoretic—abundance signal is insufficient to disambiguate sibling strains that share >99.9% of their k-mers.

### 2.11 Parallel Mapping

Read mapping is parallelized using `rayon`'s work-stealing scheduler [Burns, 2016–2024]. Reads are divided into batches, each processed by a separate thread. Within each thread, reads are mapped sequentially against all indexed genomes. This approach provides near-linear speedup with the number of available CPU cores.

## 3 Results

### 3.1 Benchmark Setup

**Three-genome benchmark:** Simulated Illumina-like reads (100 bp, 0.1% error rate, Q30–Q40) from three distantly related genomes:

- *Escherichia coli* K-12 MG1655 (4.6 Mb)
- *Staphylococcus aureus* (2.9 Mb)
- *Saccharomyces cerevisiae* S288C (12.2 Mb, 16 chromosomes)

20,000 reads total (10,000 E. coli, 5,000 S. aureus, 5,000 S. cerevisiae). Hardware: standard desktop CPU (Windows, PowerShell).

**CAMI Low Complexity benchmark [Schechter et al., 2018]:** 20,000 paired-end reads (2×150 bp, Illumina) across 62 microbial genomes (1,880 sequences, ∼1.4 GB index). The dataset includes:

- Numeric genomes (e.g., 1036554) — 8,000 reads
- Other single-contig genomes — 8,000 reads
- Sample genomes — 2,000 reads
- evo_* (>99.9% identical strains) — 16,202 reads

Setup: $k = 10$, top_n=2, 8 threads, CAMI-specific genome naming (`--cami` flag).

### 3.2 Three-Genome Classification

Table 1 shows results for mapping 20,000 simulated reads against an index containing all three genomes. The k-mer size was set to $k = 10$, and top_n=3 anchors were used.

Table 1: Three-genome benchmark: 20,000 simulated reads mapped to 3-genome index ($k = 10$, top_n=3, hybrid alignment)

| Genome | Mapped | Mapping rate | Accuracy |
|---|---|---|---|
| *E. coli* (4.6 Mb) | 9,924/10,000 | 99.2% | 99.9% |
| *S. aureus* (2.9 Mb) | 4,968/5,000 | 99.4% | 99.9% |
| *S. cerevisiae* (12.2 Mb) | 4,970/5,000 | 99.4% | 100.0% |
| **Total** | **19,862/20,000** | **99.3%** | **99.9%** |

Bit-Pop achieves 99.3% mapping rate and 99.9% classification accuracy across bacterial and eukaryotic genomes spanning different domains of life. Mapping throughput is approximately 1,500 reads/second (top_n=1) or 500 reads/second (top_n=3). Total pipeline time for 20,000 reads: 0.9s (top_n=1) to 2.8s (top_n=3) for E. coli.

### 3.3 CAMI Low Complexity Benchmark

Table 2 shows results on the CAMI Low Complexity dataset with 62 microbial genomes.

Table 2: CAMI Low Complexity benchmark: 20,000 paired-end reads across 62 genomes

| Metric | Value | Notes |
|---|---|---|
| Overall mapping rate | 92.0% | 30,105/40,000 reads |
| Overall accuracy | 85.9% | 14,222/16,566 mapped pairs |
| Time | ∼7.9s / 10k reads | 8 threads, $k = 10$, top_n=2 |
| **Breakdown by genome type** | | |
| Numeric genomes | 85.5% | ∼8,000 reads |
| Other genomes | 88.3% | ∼8,000 reads |
| Sample genomes | 91.3% | ∼2,000 reads |
| evo_* (>99.9% identical strains) | 58.4% | 16,202 reads |

The overall accuracy of 85.9% represents a significant improvement over earlier versions (71.6% with 2,500 reads). The improvement was achieved through:

1. **CAMI-specific genome naming:** The `--cami` flag extracts genome names from filenames instead of FASTA headers, fixing a naming mismatch that caused accuracy to drop to 1.07% in early versions.

2. **Preservation of evo_* .NNN suffixes:** The `.011` suffix in `evo_1049056.011` is preserved, giving each strain a unique name in the index.

3. **Increased dataset size:** 20,000 reads (vs. 2,500) provides better statistical significance.

**Paired-end conflicts:** 35.5% of read pairs have R1 and R2 mapping to different genomes, reducing effective accuracy. This is expected for reads originating from regions shared between genomes.

**The evo_* (>99.9% identical strains) challenge:** The evo_* strains share most k-mers with each other and with their parent numeric genomes. This causes reads to map to the wrong strain or to the parent genome. The top misclassifications are:

- `evo_1049056.011` → `evo_1049056.013` (45×)

- evo_1035930.029 → evo_1035930.032 (23×)
- evo_1049056.011 → evo_1049056.015 (18×)

This is a fundamental information-theoretic limitation, not an algorithmic one. The probability of a 150 bp read spanning a strain-specific SNP position is approximately $150/10^6 = 0.015\%$. This means reads from near-identical strains are informationally indistinguishable at this read length. Long reads (PacBio/Nanopore) or known SNP positions (VCF integration) would be required to surpass this barrier.

### 3.4 EM Post-Processing Results

Table 3 shows the impact of EM post-processing on the CAMI benchmark.

Table 3: EM post-processing impact on CAMI benchmark

| Metric | Baseline | + EM | Delta |
|---|---|---|---|
| Overall accuracy | 85.87% | ∼86.5% | +0.6pp |
| evo_* (>99.9% identical strains) accuracy | 58.4% | 59.8% | +1.4pp |

EM provides modest improvement (+1.4pp on evo_* (>99.9% identical strains) because abundance signals are insufficient to disambiguate sibling strains sharing >99.9% of their k-mers. The `--confidence-threshold` parameter (set to 0.95) prevents EM from breaking high-confidence correct predictions.

### 3.5 Error Tolerance

Table 4 shows mapping rate and classification accuracy across different simulated error rates (single-genome, E. coli, 1,000 reads, $k = 10$).

Table 4: Error tolerance: mapping rate and classification accuracy at different error rates

| Error rate | Mapping rate | Accuracy |
|---|---|---|
| 0.1% | 88.4% | 100.0% |
| 0.5% | 78.9% | 100.0% |
| 1.0% | 77.2% | 100.0% |
| 2.0% | 69.3% | 100.0% |
| 5.0% | 48.3% | 100.0% |
| 10.0% | 23.4% | 100.0% |

Classification accuracy remains at 100% across all error rates: when Bit-Pop maps a read, it always assigns it to the correct genome. The primary impact of sequencing errors is on mapping sensitivity (the anchor filter fails to find candidates when the rarest k-mer contains a sequencing error), not on classification precision.

### 3.6 Fuzzy K-mer Matching

The three fuzzy k-mer methods address the strain resolution challenge for evo_* (>99.9% identical strains):

Table 5: Fuzzy k-mer methods: trade-offs

| Method | Speed | Accuracy |
|---|---|---|
| Exact k-mer (baseline) | 1× | 54.2% |
| Fuzzy k-mer | ∼30× slower | Best (strain resolution) |
| Fuzzy seed | ∼20× slower | Good balance |
| Neighborhood | Fastest query | Good, larger index |

Fuzzy k-mer matching is most effective for strain-level resolution where exact k-mer matching fails. The trade-off is significant computation time ($\sim 30\times$ slower for $N = 1$), making it suitable for targeted analysis rather than high-throughput screening.

# 4 Discussion

## 4.1 Comparison with Existing Tools

Unlike Bowtie2, BWA-MEM, or minimap2, Bit-Pop does not attempt to produce a full alignment for every read. Instead, it focuses on the classification task: identifying which genome in a collection best explains each read. This focus enables several advantages:

- **Compact databases:** Database sizes are proportional only to the user's genomes (MBs), not entire taxonomic databases (100 GB+).
- **Offline operation:** No internet connectivity required.
- **Fast updates:** Adding a new genome takes seconds (rebuild index), not hours/days (rebuild database).
- **Unified ranking:** Automatic genome assignment with confidence scores.

Table 6 summarizes key feature differences between Bit-Pop and established tools.

Table 6: Feature comparison with established genomic tools

| **Feature** | **Bit-Pop** | **Bowtie2** | **BWA-MEM** | **minimap2** |
|---|---|---|---|---|
| Multi-genome classification | ✓ | ✗ | ✗ | ✗ |
| Compact database (MBs) | ✓ | ✗ | ✗ | ✗ |
| Offline operation | ✓ | ✓ | ✓ | ✓ |
| Single-genome alignment | ✓ | ✓ | ✓ | ✓ |
| SAM output | ✓ | ✓ | ✓ | ✓ |
| Paired-end support | ✓ | ✓ | ✓ | ✓ |
| Long-read support | ✓ | ✗ | ✓ | ✓ |
| Splice-aware mapping | ✗ | ✗ | ✗ | ✓ |
| Pangenome graph | ✗ | ✗ | ✗ | ✓ |
| NCBI integration | ✓ | ✗ | ✗ | ✗ |
| Fuzzy k-mer matching | ✓ | ✗ | ✗ | ✗ |

Bit-Pop's unique capability—simultaneous mapping across thousands of genomes with automatic ranking using compact databases—is not offered by any existing general-purpose aligner. This makes it particularly suitable for species classification tasks where the goal is to identify the source organism rather than produce a detailed per-base alignment.

## 4.2 Comparison with Kraken2

Kraken2 [Wood et al., 2019] is designed for broad-spectrum classification against entire taxonomic databases, requiring 100 GB+ storage and hours to days for database construction. This design is optimal for "unknown unknown" scenarios where the target organism is not predetermined. Bit-Pop addresses the complementary use case: targeted classification against a user-defined genome collection, with database sizes proportional only to the genomes of interest (MBs, not GBs), no internet connectivity requirement, and index construction in minutes. A direct runtime and accuracy comparison was not performed in this work, as Kraken2 requires the full NCBI taxonomy hierarchy even for small custom genome sets, making equivalent experimental conditions impractical at this stage.

Table 7: Bit-Pop vs Kraken2: key differences

| Aspect | Bit-Pop | Kraken2 |
|---|---|---|
| Database size | MB (only your genomes) | 100 GB+ (entire NCBI) |
| Internet required | No | Yes (every update) |
| Build time | 2 minutes | Hours to days |
| Offline operation | Full | No |
| Custom update | Seconds (add 1 genome) | Rebuild entire database |

Kraken2 is better for broad metagenomics where you do not know what you are looking for. Bit-Pop is better for **targeted searching** where you know what matters.

## 4.3 Limitations

Several limitations should be noted:

- **Strain-level resolution:** Genomes that are >99.9% identical (same sample, different strains) share most k-mers. Reads may map to the wrong strain or to a parent genome. This is a fundamental information-theoretic limitation, not an algorithmic one. Empirical analysis on the CAMI Low dataset showed that reads from sibling evo_* (>99.9% identical strains) (e.g., `evo_1049056.015` vs `evo_1049056.011`) are misassigned bidirectionally with near-equal frequency, confirming that the alignment signal does not carry sufficient discriminative information at 150 bp read length. The probability of a 150 bp read spanning a strain-specific SNP position is approximately $150/10^6 = 0.015\%$, meaning the vast majority of reads from near-identical strains are informationally indistinguishable. EM post-processing improved evo_* (>99.9% identical strains) accuracy by +1.4 pp (58.4% → 59.8%), but population-level abundance signals were insufficient for reliable sibling strain disambiguation. Long reads (PacBio/Nanopore) or known SNP positions (VCF integration) would be required to surpass this barrier.
- **Proof-of-concept stage:** This work presents benchmarks on simulated reads and the CAMI Low Complexity dataset. The tool has not been validated on large-scale real datasets or compared directly with established aligners.
- **Mapping rate at high error rates:** While the top-$N$ anchor filter provides 99.3% mapping rate at 0.1% error rate, performance degrades to 23.4% at 10% error rate.
- **Paired-end conflicts:** 35.5% of read pairs have R1 and R2 mapping to different genomes, reducing effective accuracy.
- **No clinical validation:** Bit-Pop is an academic research tool and has not been validated for clinical or diagnostic use.
- **Index file sizes:** ∼152 MB for 19.7 Mb genome collection. Delta compression is planned to reduce this by 5–10×.

## 4.4 Future Work

Planned improvements include:

- **SNP-aware weighting:** Incorporate known SNP positions (VCF files) to differentiate near-identical strains.
- **Multi-k consensus:** Combine results from multiple k-mer sizes ($k = 8$, $k = 10$, $k = 12$) for improved strain resolution.
- **Long-read support:** Extend to PacBio and Nanopore reads with higher error rates.

- **SIMD acceleration:** AVX2/AVX-512 acceleration of the 2-bit XOR alignment pipeline.
- **SA compression:** Delta-VLI compression of suffix arrays to reduce index file sizes by 5–10×.
- **Direct comparison:** Benchmark against Bowtie2, BWA-MEM, and Kraken2 on multi-genome classification tasks.
- **Larger benchmarks:** 100+ genomes and eukaryotic genomes.

# 5 Conclusion

Bit-Pop provides a solution for multi-genome DNA read classification that combines speed, compactness, and accuracy. By combining FM-index-based top-$N$ k-mer filtering with reverse complement support, bit-level XOR alignment (2.3 ns per 31-base chunk), Myers edit distance, and Smith–Waterman refinement, it achieves 99.3% mapping rate and 99.9% classification accuracy across bacterial and eukaryotic genomes. On the CAMI Low Complexity benchmark with 62 microbial genomes, Bit-Pop achieves 92.0% mapping rate and 85.9% overall accuracy. The evo_* (>99.9% identical strains) remain challenging at 59.8% accuracy (with EM refinement), representing a fundamental information-theoretic limitation of short-read k-mer-based classification. Bit-Pop is designed as a focused tool for species classification, contamination detection, and targeted metagenomic analysis where users work with known genome collections, filling a niche not addressed by existing general-purpose aligners.

# Availability

Source code is available at `https://github.com/mladenpop-oss/bit-pop` under the MIT License. Benchmark scripts and example datasets are included in the repository. A Zenodo archive with a citable digital object identifier (DOI) is available at `https://doi.org/10.5281/zenodo.20043593`.

# References


B. Burns. Rayon: Data parallelism in rust. `https://github.com/rayon-rs/rayon`, 2016–2024.

M. Burrows and D. J. Wheeler. A block-sorting lossless data compression algorithm. In *HP Technical Report*, number 124, 1994.

B. Ewing and P. Green. Base-calling of automated sequencer traces using phred. ii. error probabilities. *Genome Research*, 8(3):186–194, 1998. doi: 10.1101/gr.8.3.186.

E. M. Ferrada and E. V. Grossini. Efficient exact suffix arrays and related indexes for string collections. *Theoretical Computer Science*, 387(2):131–144, 2007. doi: 10.1016/j.tcs.2007.07.020.

J. Kärkkäinen and P. Sanders. Simple suffix sorting. *Journal of Discrete Algorithms*, 1(2):235–257, 2003. doi: 10.1016/S1570-8667(03)00006-3.

D. Kim, L. Song, F. P. Breitwieser, and S. L. Salzberg. Centrifuge: rapid and accurate metagenomic sequence classification using randomised permutations. *Nature Biotechnology*, 34:587–589, 2016. doi: 10.1038/nbt.3589.

B. Langmead and S. L. Salzberg. Fast gapped-read alignment with bowtie 2. *Nature Methods*, 9(4):357–359, 2012. doi: 10.1038/nmeth.1923.

H. Li. Aligning sequence reads, clone sequences and assembly contigs with bwa-mem. *Bioinformatics*, 29 (22):2930–2941, 2013. doi: 10.1093/bioinformatics/btt509.

H. Li. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, 2018. doi: 10.1093/bioinformatics/bty191.

G. Myers. A fast bit-vector algorithm for approximate string matching based on dynamic programming. *Journal of Computer and System Sciences*, 59(3):396–415, 1999. doi: 10.1006/jcss.1999.1652.

M. S. Schechter, J. P. Meier-Kolthoff, T. Woyke, and N. C. Kyrpides. Cami: a benchmark community for metagenomic tools. *Nature Biotechnology*, 36:113–114, 2018. doi: 10.1038/nbt.4276.

T. F. Smith and M. S. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1):195–197, 1981. doi: 10.1016/0022-2836(81)90087-5.

Patrik Sósten. libsais: Library for integer array suffix sorts. `https://github.com/isawalls/libsais`, 2020–2024.

D. E. Wood, J. Lu, and B. Langmead. Improved metagenomic analysis with kraken 2. *Genome Biology*, 20 (1):257, 2019. doi: 10.1186/s13059-019-1891-0.